算法效率度量

2025年3月16日 22:08

时间复杂度：重复执行次数f(n) 是 问题规模n 的函数

空间复杂度：所需存储空间f(n) 是 问题规模n 的函数

线性表

2025年3月16日 22:14

名词：

数据项：一个元素的不同属性

记录：也就是元素

文件：有大量记录的线性表

线性表的顺序表示：

相邻元素地址相同

线性表的链式表示：

分类：单链表、双链表（如双向链表）、多链表、循环链表、静态链表等

线性链表：

python中可以用class表示节点，如：

```python
class Node:
    def __init__(self, data = None, the_next = None):
        self.data = data
        self.the_next = the_next
node_1 = Node(1)
node_2 = Node(2)
node_3 = Node(3)
node_4 = Node(4)
node_1.the_next, node_2.the_next, node_3.the_next = node_2, node_3, node_4
print(node_1.the_next.the_next.data)
```

可以使用头节点，数据域存链表信息（长度等），指针指向链表中第一项

静态链表：

用结构体数组的方式存储链表，链表表项的指针域不是地址，而是数组下标

i	data	cur
0	头结点	1
1	ZHAO	3
2	LI	5
3	QIAN	7
4	WU	0
5	ZHOU	4
6	SUN	2
7	LIU	6
1000	…	…

代码例子：

```python
class Node:
    def __init__(self, data = None, cur = None):
        self.data = data
        self.cur = cur
MAXSIZE = 5     #规定静态链表的规模
static_linklist = [None] * MAXSIZE
static_linklist[0] = Node('a', 3)
static_linklist[1] = Node('b', None)
static_linklist[2] = Node('c', 1)
static_linklist[3] = Node('d', 2)
i = 0
while i != None:
    print(f'{i}: ' + static_linklist[i].data)
    i = static_linklist[i].cur
#在链表元素c后面插入元素
i = 0
while static_linklist[i].data != 'c':
    i = static_linklist[i].cur
static_linklist[4] = Node('new', static_linklist[i].cur)
static_linklist[i].cur = 4

i = 0
while i != None:
    print(f'{i}: ' + static_linklist[i].data)
    i = static_linklist[i].cur
```

栈（stack）/队列（queue）

2025年3月18日 9:48

栈（stack）的定义：

栈顶（top）：出栈入栈的地方（有的定义为指向第一个有数据的项，有的为指向第一个有数据的项上面的空项）

栈底（base）

特点：后进先出（LIFO）

表示：

顺序栈：用数组的方式实现

base = NULL	栈不存在
top = base	空栈
top = stacksize	满栈

链栈：以链表的方式实现

队列（queue）的定义：

队头（front）：出队的地方

队尾（rear）：入队的地方

特点：先进先出（FIFO）

表示：

顺序队列：用数组的方式实现，但入队出队会导致front和rear往数组后面移动，数组前面的空间废弃

front = rear	空队列

链队列：用链表的方式实现，不会导致前面的空间废弃，无法估计队列规模用链队列

循环队列：数组最后一项和第一项相连（比如rear和front模N，N为队列规模），但是导致无法区分空队列和满队列（都是front = rear）

解决方法1	设置标志位区分空满
解决方法2	空一个元素，front = rear则为空，rear + 1 = front 则为满

串

2025年3月18日 10:19

串的模式匹配算法（确定主串所含子串第一次出现的位置）

1.BF算法（带回溯，速度慢）：

将主串S的第pos个字符和模式T的第一个字符比较

若相等，继续逐个比较后续字符

若不等，从主串S的下一个字符（pos+1）重新与T的第一个字符比较

直到S的一个连续字串与模式T相等，返回S中与T匹配的子序列的第一个字符的序号

性能分析：

主串长n；子串长m。可能匹配成功的位置（1 ~ n-m+1）。

①最好的情况下：

第i个位置匹配成功，前i-1个位置每个位置只比较了一次，在第i个位置比较了m次，所以第i个位置共比较了（i-1+m）次，平均比较次数：

S="0000001111"
T="1111"

$$\sum_{i=1}^{n-m+1} p_i(i-1+m)=\frac{1}{n-m+1}\sum_{i=1}^{n-m+1}(i-1+m)=\frac{1}{2}(m+n)$$

最好情况下算法的平均时间复杂度O(n+m)。

②最坏的情况下：

第i个位置匹配成功，但每个i-1位置都比较了m次，总共比较了(i*m)次，平均比较次数：

S="0000000001"
T="0001"

$$\sum_{i=1}^{n-m+1} p_i(i\times m)=\frac{m}{n-m+1}\sum_{i=1}^{n-m+1} i=\frac{1}{2}m(n-m+2)$$

设n>m，最坏情况下的平均时间复杂度为O(n*m)。

2.KMP算法（避免回溯）:**时间复杂度O(n+m)**

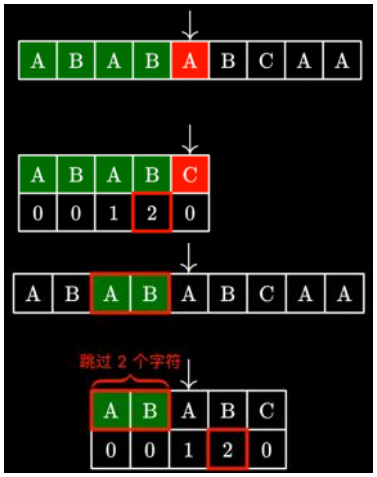
观察模式T，如果有公共前后缀（T中开头和结尾有一样的字符串，如ABCAB，公共前后缀是AB），下次匹配失败时不避回溯到主串S的下一个字符（pos+1），因为主串匹配失败处前面的字符和字串的前缀匹配

算法实现：

1.主串S的指针永不回溯

2.求模式T的最长公共前后缀，构造next数组

3.匹配失败时查询模式T失配处前一个字符对应的next数组值，这是重新开始匹配时跳过的模式T的字符数



用递推方法求Next数组：

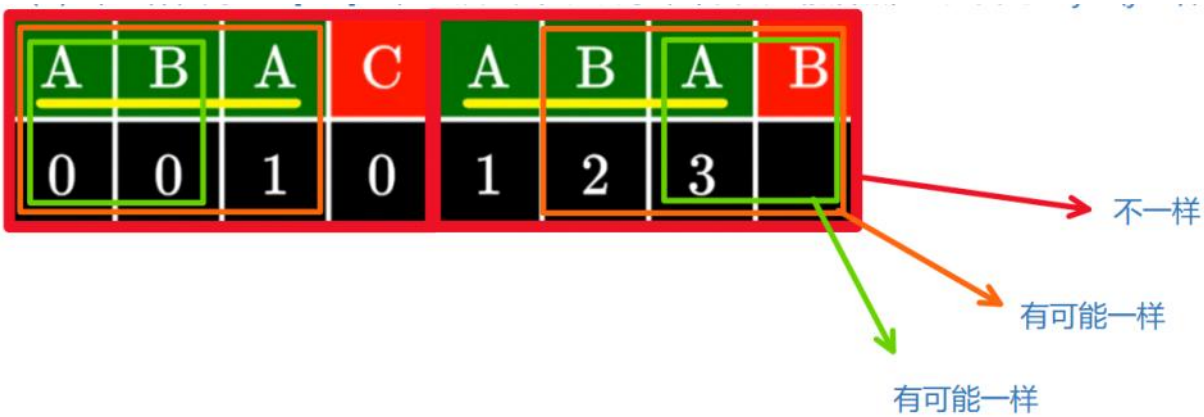
1.next[i]表示当前模式T的0~i-1字符组成的字串中的最长公共前后缀（必须小于i，没有就是0）

2.next[0]=0

3.假设已经知道next[i]=x（也就是T的0~x-1和i-x+1~i字符是一样的），则对于下一个字符（x和i+1比较）：

（1）下一字符也一样，则next[i+1]=x+1

（2）不一样，则next[i+1]≤x，也就是在更短的子串中找公共前后缀，比如找到0~y-1(y<x)和i-y+1~i字符相同



实际上可以把0~x-1和i组合在一起继续寻找公共前后缀（因为0~x-1和i-x+1~i字符是一样的）

那么继续看B前面的Anext值，也就是next[x-1]的值，在此基础上比较下一个字符（next[x-1]和i+1是否相等）

递推法求next数组的程序描述为：

next[0]=0

对于第i字符，如果第next[i-1]字符和第i字符相同，next[i]=next[i-1]+1

如果不同，把上面的next[i-1]换成next[next[i-1]-1]，看第next[next[i-1]-1]字符和第i字符是否相同，不同继续（值-1作为next下标，拿到next值），直到比较到第1和第i个字符，没

有则next[i]=0

代码：

```python
def compute_next(pattern):
    next_array = [0] * len(pattern)
    j = 0  # j表示最长公共前后缀的长度

    for i in range(1, len(pattern)):
        #如果当前字符不匹配，寻找更短的公共前后缀，直到0
        while j > 0 and pattern[i] != pattern[j]:
            # 如果当前字符不匹配，且j不为0，更新j的值
            j = next_array[j - 1]

        # 如果当前字符匹配，更新j的值
        if pattern[i] == pattern[j]:
            j += 1

        next_array[i] = j

    return next_array

def KMP(my_str, pattern):
    next_array = compute_next(pattern)
    pos_my_str = 0
    pos_pattern = 0

    while pos_my_str <= len(my_str):
        if pos_pattern == len(pattern):
            return pos_my_str - len(pattern)
        if my_str[pos_my_str] == pattern[pos_pattern]:
            pos_my_str += 1
            pos_pattern += 1
        else:
            if pos_pattern == 0:
                pos_my_str += 1
            else:
                pos_pattern = next_array[pos_pattern - 1]
    return -1

if __name__ == "__main__":
    my_str = "aaaacbbaabcc"
    pattern = "aabcc"
    pos = KMP(my_str, pattern)
    print(pos)
```

矩阵压缩

2025年3月25日 9:19

定义：

特殊矩阵：相同元素或0元素在矩阵中有分布规律，如对称矩阵、对角矩阵、三角矩阵

稀疏矩阵：分布不规律（不为0的元素占比5%以下）

用三元组表示稀疏矩阵：（i, j, a_{ij}）

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

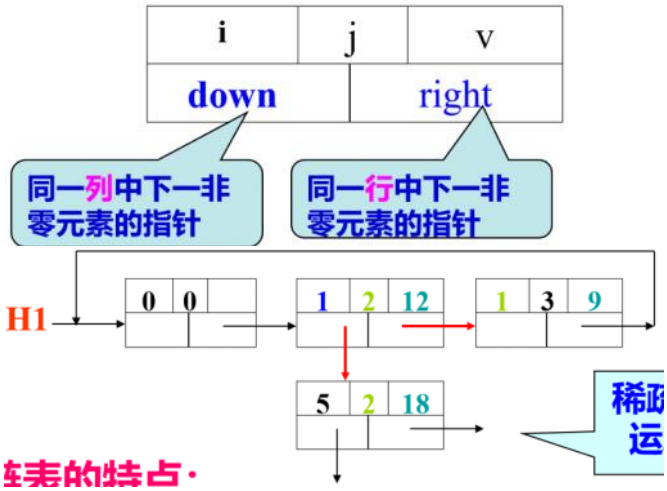
稀疏矩阵压缩存储的缺点：失去随机存取功能

三元组表示方法

线性表三元组：

((1,2,12) , (1,3,9), (3,1,-3), (3,5,14), (4,3,24), (5,2,18) , (6,1,15), (6,4,-7))

十字循环链表三元组：



三元组矩阵：

	i	j	value
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

法4：用带辅助向量的三元组表示。

用途：**便于高效访问稀疏矩阵中任一非零元素。**

方法：增加2个辅助向量：

① **记录每行非0元素个数，用NUM（i）表示；**

② **记录稀疏矩阵中每行第一个非0元素在三元组中的行号，用POS（i）表示。**

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

	i	j	v
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

i	1	2	3	4	5	6
NUM(i)	2	0	2	1	1	2
POS(i)	1	3	3	5	6	7

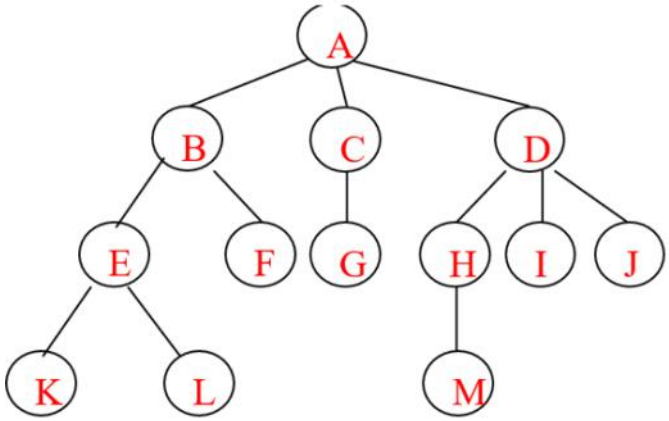
POS（i）如何计算？

POS(1)＝1

POS(i)＝POS(i-1)+NUM(i-1)

基本术语与性质

2025年3月25日 10:05



前驱：双亲
后继：孩子
根节点：无前驱
叶子节点：终端节点，无后继，度为0
兄弟：同一双亲的孩子
有序树，无序树

节点的度：节点的子树数
节点的层次：从根节点到该节点的层数（根节点算第一层）
树的度：所有节点中度的最大值
树的深度（高度）：所有节点中的最大层数

右上图：节点数13，树的度：3，树的深度：4

1. 节点数n=所有节点的度+1（根节点）
2. 度为m的树第i层最多m^{i-1}节点
3. 高为h、度为m的树最多n=$\frac{m^h-1}{m-1}$节点（等比数列求和）
4. n节点、度为m的树最小高度h= 向上取整[$\log_m(n(m-1))+1$]（性质三反过来）

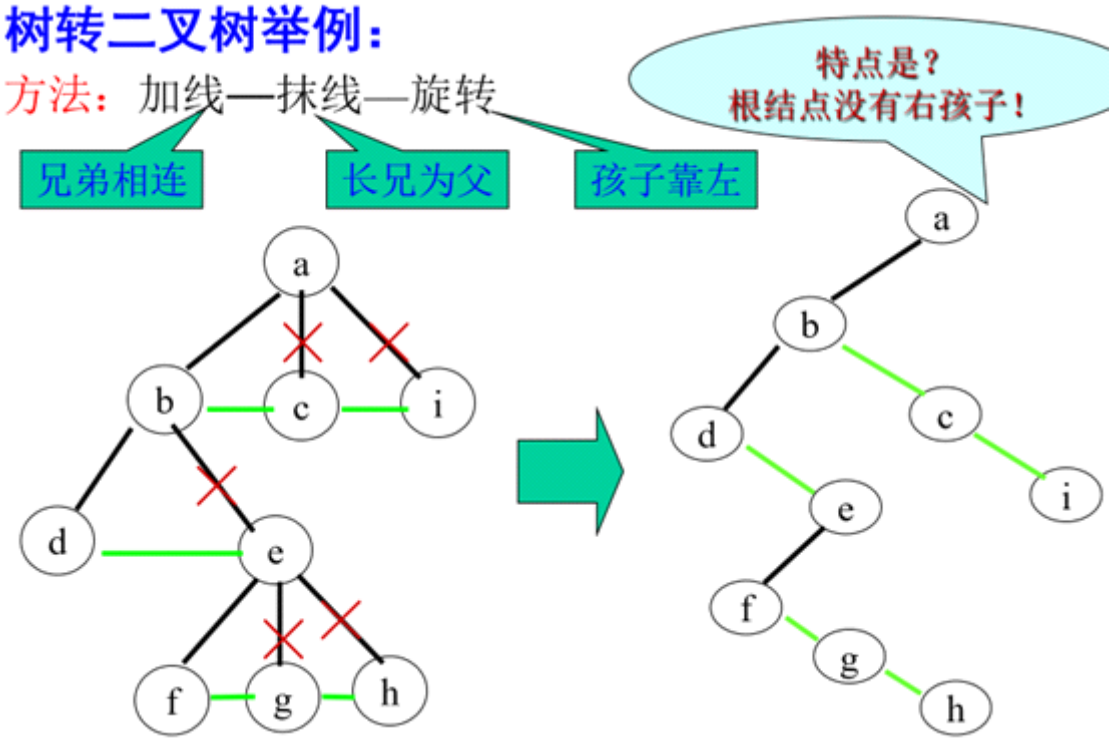
二叉树深度

```python
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def maxDepth(Node):
    if Node is None:
        return 0
    else:
        depth_left = maxDepth(Node.left)
        depth_right = maxDepth(Node.right)
    return max(depth_left, depth_right) + 1
```

树转二叉树

2025年3月25日 10:16

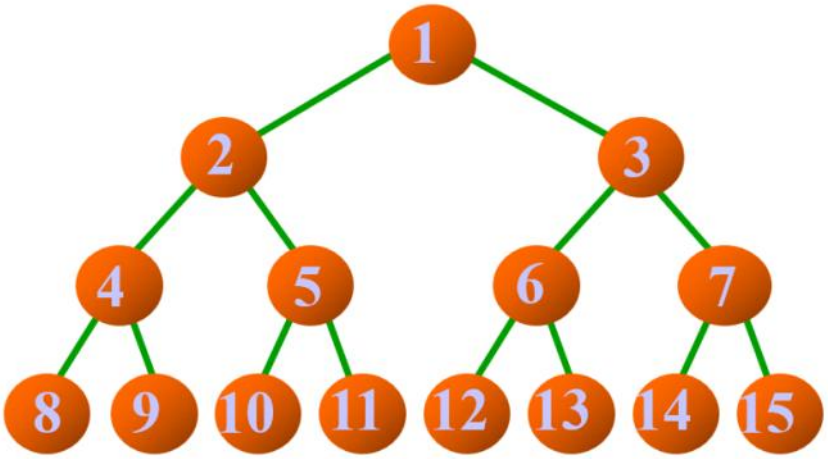
加线（兄弟相连）——抹线（长兄为父）——旋转（孩子靠左）



树的存储

2025年3月25日 10:25

满二叉树：深度为k有2^k-1个节点



完全二叉树：深度为k，除第k层外都满，第k层从右往左连续缺若干节点

完全二叉树编号性质(完全二叉树的顺序存储)

若i=1，根节点否则 向下取整[i/2]为其父节点

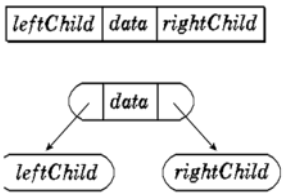
2i为i的左孩子，2i+1为其右孩子（不超过总节点数）

非完全二叉树的顺序存储：添加空节点补成完全二叉树

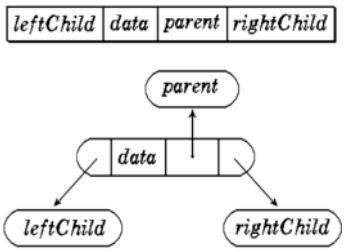
二叉树的遍历

2025年3月25日 15:26

二叉树链表的遍历：



(a) 二叉链表



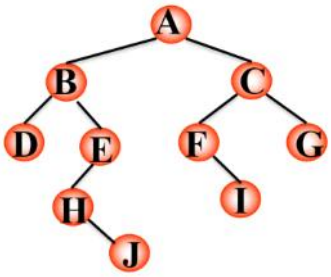
(b) 三叉链表

先序遍历DLR：节点→左→右

中序遍历LDR：左→节点→右

后序遍历LRD：左→右→节点

递归算法示例

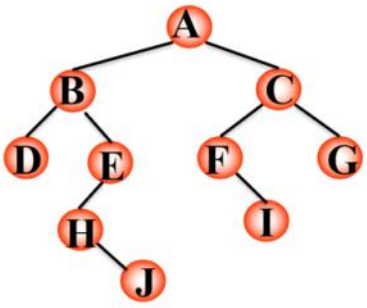


二叉树前中后序遍历

```python
class Node:
    def __init__(self, Node_Data = None, L_child = None, R_child = None):
        self.Node_Data = Node_Data
        self.L_child = L_child
        self.R_child = R_child

def DLR(node):
    if node != None:
        print(node.Node_Data, end=" ")
    if node.L_child != None:
        DLR(node.L_child)
    if node.R_child !=None:
        DLR(node.R_child)

def LDR(node):
    if node.L_child != None:
        LDR(node.L_child)
    if node != None:
        print(node.Node_Data, end=" ")
    if node.R_child !=None:
        LDR(node.R_child)

def LRD(node):
    if node.L_child != None:
        LRD(node.L_child)
    if node.R_child !=None:
        LRD(node.R_child)
    if node != None:
        print(node.Node_Data, end=" ")

Root_Node = Node("A", Node("B", Node("D"), Node("E", Node("H", R_child= Node("J")))), Node("C", Node("F", R_child= Node("I")), Node("G")))
DLR(Root_Node)
print("\n", end="")
LDR(Root_Node)
print("\n", end="")
LRD(Root_Node)
```

非递归算法使用栈：

非递归的二叉树前序遍历：



1）建立栈**stack**, 初始时栈为空
2）**t**指向根
3）当 **t**不为空时，或栈**stack**不空时，重复：
若**t**不空，访问**t→data**后，将**t**入栈；
t指向其左子女；
否则：栈顶元素出栈；
t指向其右子女
4）结束

访问A,	A进栈，t指向B	栈[A]
访问B,	B进栈，t指向D	栈[AB]
访问D,	D进栈，t为空	栈[ABD]
	D出栈，t为空	栈[AB]
	B出栈，t指向E	栈[A]
访问E,	E进栈，t指向H	栈[AE]
访问H,	H进栈，t为空	栈[AEH]
	H出栈，t指向J	栈[AE]
访问J,	J进栈，t为空	栈[AEJ]
	J出栈，t为空	栈[AE]
	E出栈，t为空	栈[A]
	A出栈，t指向C	栈为空
访问C,	C进栈，t指向F	栈[C]
访问F,	F进栈，t为空	栈[CF]
	F出栈，t指向I	栈[C]
访问I,	I进栈，t为空	栈[CI]
	I出栈，t为空	栈[C]
	C出栈，t指向G	栈为空
访问G,	G进栈，t为空	栈[G]
	G出栈，t为空	栈为空

二叉树的重建

2025年3月25日 15:26

前序和中序，可唯一地确定一棵二叉树

后序和中序，可唯一地确定一棵二叉树

前序和后序不可唯一地确定一棵二叉树

```python
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def rebuild_from_DLR_LDR(DLR, LDR):
    if not DLR:
        return None
    if len(DLR) ==1:
        return TreeNode(DLR[0])
    return TreeNode(DLR[0], rebuild_from_DLR_LDR(DLR[1 : LDR.index(DLR[0]) + 1], LDR[ : LDR.index(DLR[0])]), rebuild_from_DLR_LDR(DLR[LDR.index(DLR[0]) + 1 : ], LDR[LDR.index(DLR[0]) + 1 : ]))

def DLR(node):
    if node is None:
        return
    else:
        print(node.val, end=" ")
        if node.left is not None:
            DLR(node.left)
        if node.right is not None:
            DLR(node.right)

def LDR(node):
    if node is None:
        return
    else:
        if node.left is not None:
            LDR(node.left)
        print(node.val, end=" ")
        if node.right is not None:
            LDR(node.right)

my_DLR = ["A", "B", "D", "E", "H", "J", "C", "F", "I", "G"]
my_LDR = ["D", "B", "H", "J", "E", "A", "F", "I", "C", "G"]
rebuild_tree = rebuild_from_DLR_LDR(my_DLR, my_LDR)
DLR(rebuild_tree)
print("\n", end="")
LDR(rebuild_tree)
print("\n", end="")
```

重建二叉树

树的遍历

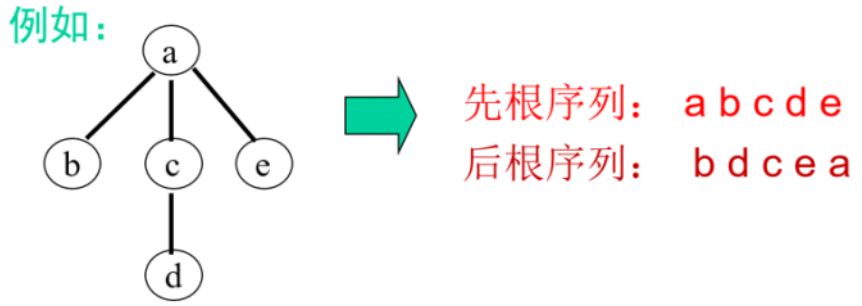
2025年3月27日 7:41

深度优先遍历（不分左右子树）

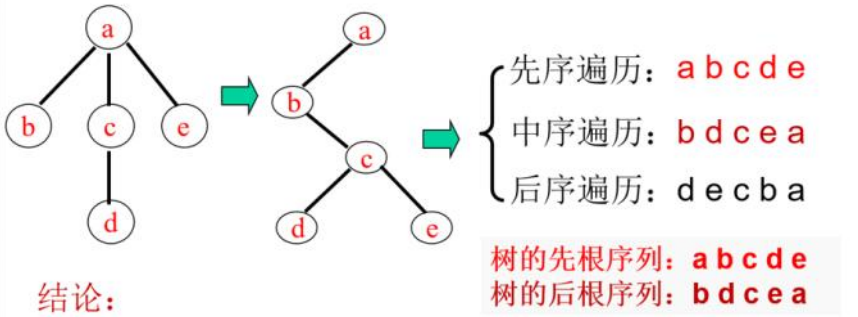
先根

后根

例如：



讨论：树若采用“先转换，后遍历”方式，结果是否一样？



结论：

1. 树的先根遍历与二叉树的先序遍历相同；
2. 树的后根遍历相当于二叉树的中序遍历；

哈夫曼树

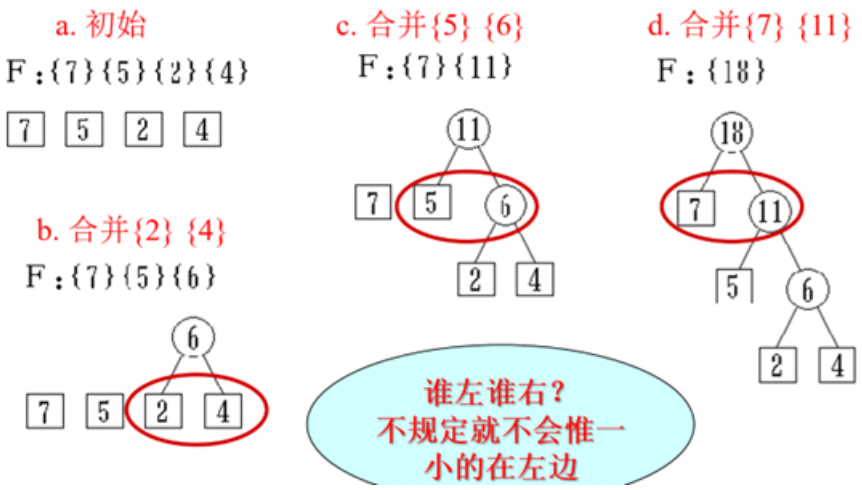
2025年3月27日 7:44

带权路径长度最小的树：

具体操作步骤：

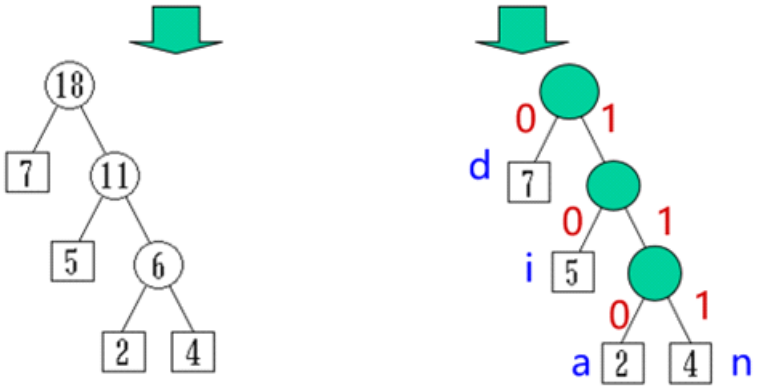
对权值进行合并、删除与替换

——在权值集合{7,5,2,4}中，总是合并当前值最小的两个权



Huffman编码

按左“0”右“1”对Huffman树的所有分支编号



Huffman编码结果：d=0，i=10，a=110，n=111

WPL=1bit×7+2bit×5+3bit×(2+4)=35

（小于等长码的WPL=36）

最小生成树

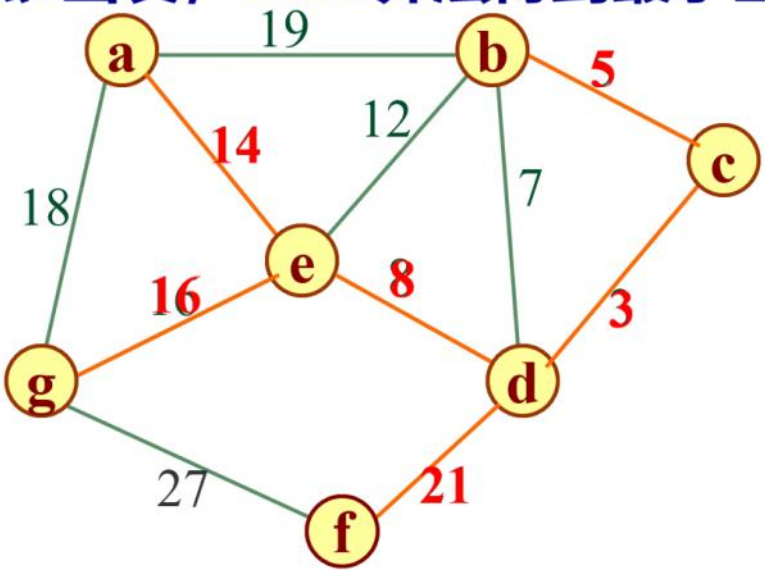
2025年3月29日 20:36

从某个顶点出发，找到权值和最小的树（不唯一）

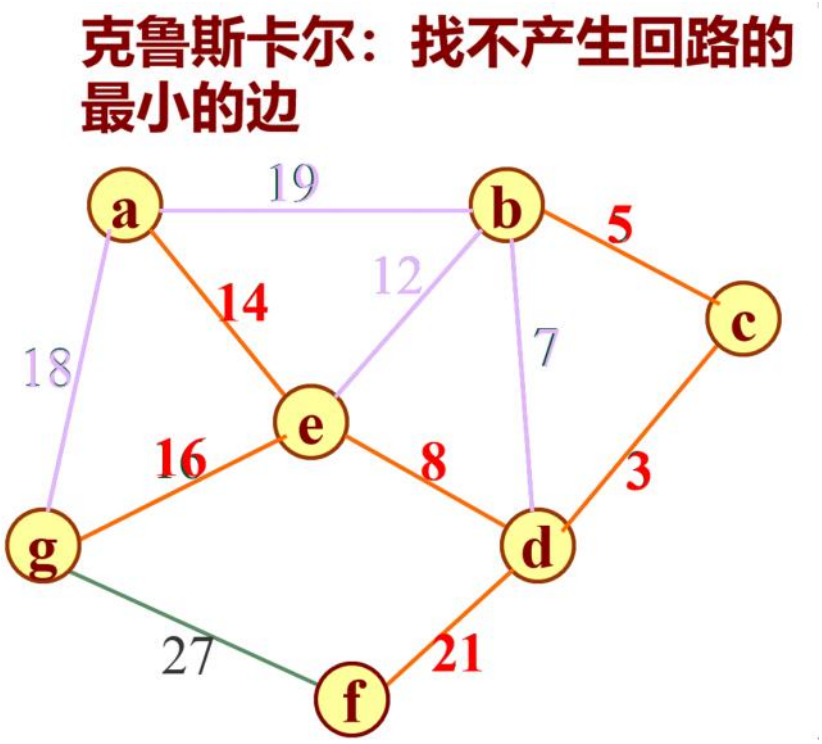
Prim （普里姆）算法将顶点归并，与边数无关，适于稠密网。

Kruskal （克鲁斯卡尔）算法：将边归并，适于求稀疏网的最小生成树。

从顶点a出发，Prim算法得到最小生成树。



从现有的顶点集合找下一条最短的边



找不构成回路的最短的线

图的基本术语

2025年3月27日 19:28

有向图

无向图

完全图：每两个顶点都有一条边

稀疏图

稠密图

子图（顶点和边都是子集）

连通图：无向图中任何两个顶点都连通

强连通图：有向图中任何两个顶点都连通

生成树：极小连通子图，n个顶点，n-1条边（多一条边必成环，少一条边必成非连通图）

弧头：有向图中边(弧)的终点

弧尾：有向图中边(弧)的起点

度：与顶点相连的边的条数

出度

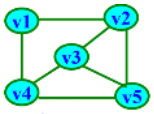
入度

图的存储结构

2025年3月27日 19:43

邻接矩阵表示法（稠密图用）

无向图



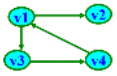
$$
\begin{array}{c|ccccc}
 & v1 & v2 & v3 & v4 & v5 \\
\hline
v1 & 0 & 1 & 0 & 1 & 0 \\
v2 & 1 & 0 & 1 & 0 & 1 \\
v3 & 0 & 1 & 0 & 1 & 1 \\
v4 & 1 & 0 & 1 & 0 & 1 \\
v5 & 0 & 1 & 1 & 1 & 0
\end{array}
$$

特点：

对称矩阵

完全图除了对角元素为0，其余全为1

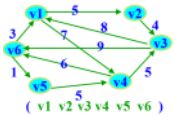
有向图



$$
\begin{array}{c|cccc}
 & v1 & v2 & v3 & v4 \\
\hline
v1 & 0 & 1 & 1 & 0 \\
v2 & 0 & 0 & 0 & 0 \\
v3 & 0 & 0 & 0 & 1 \\
v4 & 1 & 0 & 0 & 0
\end{array}
$$

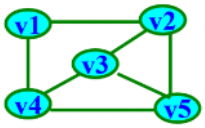
行出列入

有权图

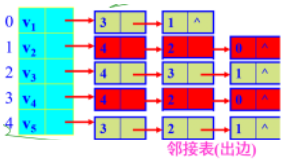


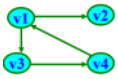
$$
\begin{array}{c|cccccc}
 & v1 & v2 & v3 & v4 & v5 & v6 \\
\hline
v1 & \infty & 5 & \infty & 7 & \infty & \infty \\
v2 & \infty & \infty & 4 & \infty & \infty & \infty \\
v3 & 8 & \infty & \infty & \infty & \infty & 9 \\
v4 & \infty & \infty & 5 & \infty & \infty & 6 \\
v5 & \infty & \infty & \infty & 5 & \infty & \infty \\
v6 & 3 & \infty & \infty & \infty & 1 & \infty
\end{array}
$$

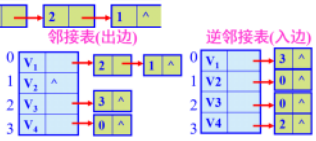
邻接表（链式）表示法（稀疏图用）



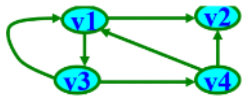
数字表示数组中的索引，比如3是v_4

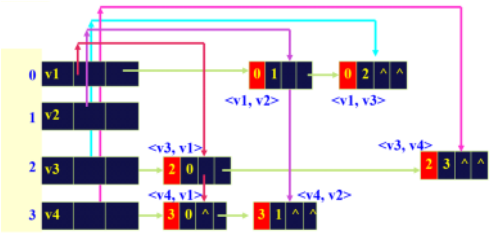






十字链表表示法：





顶点节点

顶点数据	从这里遍历可以找到以该顶点为弧头的所有弧	可遍历找到所有的以该顶点为弧尾的所有弧

弧节点

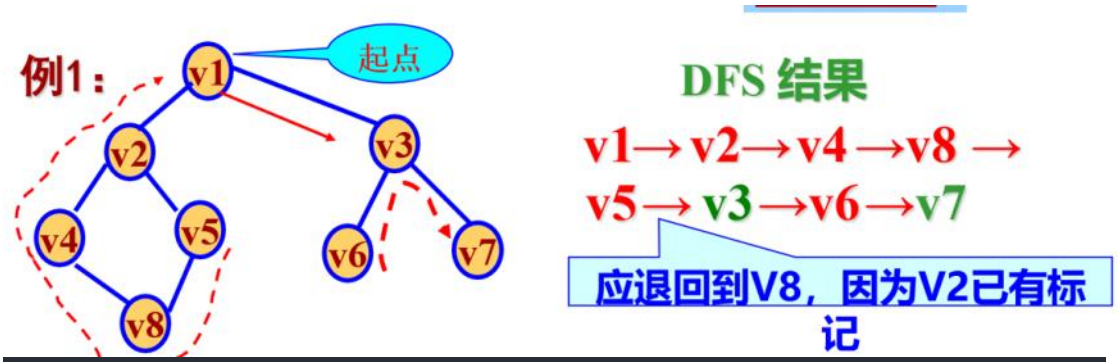
弧尾	弧头	弧头相同的下一个弧节点	弧尾相同的下一个弧节点

图的遍历

2025年3月27日 20:12

每个节点只能被访问一次（设置辅助数组visited）

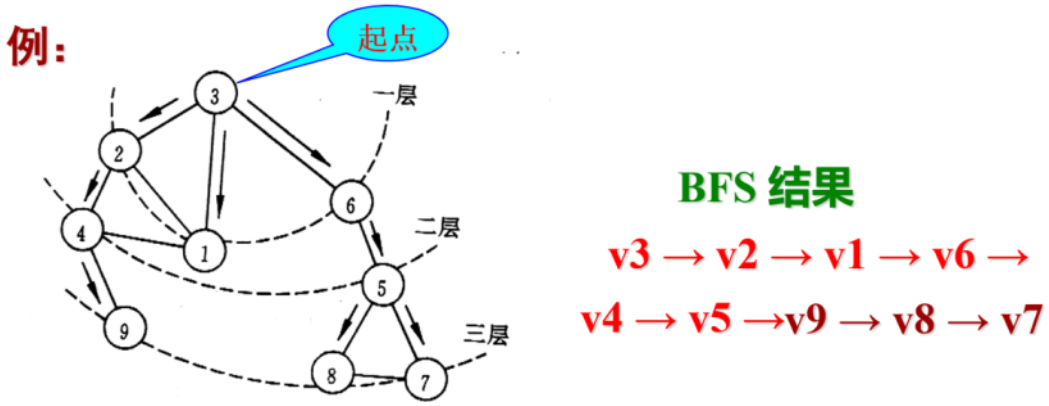
深度优先搜索(DFS)




```python
def undirected_DFS(value, near_matrix, visited, start=  0):
    print(value[start], end=" ")
    visited[start] = 1
    for i in range(len(value)):
        if near_matrix[start][i] != 0 and visited[i] != 1:
            undirected_DFS(value, near_matrix, visited, i)

value = ["v1", "v2", "v3", "v4", "v5", "v6", "v7", "v8"]
near_matrix = [[0, 1, 1, 0, 0, 0, 0, 0],
               [1, 0, 0, 1, 1, 0, 0, 0],
               [1, 0, 0, 0, 0, 1, 1, 0],
               [0, 1, 0, 0, 0, 0, 0, 1],
               [0, 1, 0, 0, 0, 0, 0, 1],
               [0, 0, 1, 0, 0, 0, 0, 0],
               [0, 0, 1, 0, 0, 0, 0, 0],
               [0, 0, 0, 1, 1, 0, 0, 0]]
visited = [0] * len(value)
undirected_DFS(value, near_matrix, visited, start= 0)
```

深度优先搜索

广度优先搜索(BFS)



```python
def undirected_DFS(value, near_matrix, visited, start=  0):
    print(value[start], end=" ")
    visited[start] = 1
    if 0 in visited:
        for i in range(len(value)):
            if near_matrix[start][i] != 0 and visited[i] != 1:
                undirected_DFS(value, near_matrix, visited, i)

value = ["v1", "v2", "v3", "v4", "v5", "v6", "v7", "v8"]
near_matrix = [[0, 1, 1, 0, 0, 0, 0, 0],
               [1, 0, 0, 1, 1, 0, 0, 0],
               [1, 0, 0, 0, 0, 1, 1, 0],
               [0, 1, 0, 0, 0, 0, 0, 1],
               [0, 1, 0, 0, 0, 0, 0, 1],
               [0, 0, 1, 0, 0, 0, 0, 0],
               [0, 0, 1, 0, 0, 0, 0, 0],
               [0, 0, 0, 1, 1, 0, 0, 0]]
visited = [0] * len(value)
undirected_DFS(value, near_matrix, visited, start= 0)
```

广度优先搜索

哈希表

2025年3月29日 20:47

简历值与储存地址的联系，查找速度O（1）

构造方法：

1.直接定址法：值和地址是线性（ax+b）关系

2.除留余数法：求余的p要≤哈希表长n

3.数字分析法：如取尾号

4.平方取中法

5.折叠法：几位几位分别相加

适用于：每一位上各符号出现概率大致相同的情况。

法1：移位法 —— 将各部分的最后一位对齐相加。

法2：间界叠加法——从一端向另一端沿分割界来回折叠后，最后一位对齐相加。

例：元素42751896, 用法1： 427 + 518 + 96=1041

用法2： 427 518 96—> 724+518+69 =1311

冲突方法：

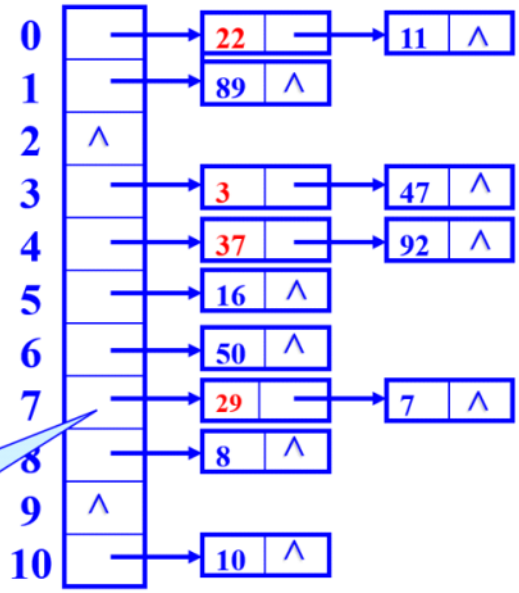
1.开放定址法：这个地址不行往后一位看看还空不空

2.链地址法：

例：设{ 47, 7, 29, 11, 16, 92, 22, 8, 3, 50, 37, 89 ,10}的哈希函数为：

Hash(key)=key mod 11，

用拉链法处理冲突，则建表如右图所示。



有冲突的元素可以插在表尾,也可以插在表头。

内部排序算法

2025年3月29日 20:58

各种内部排序方法的比较 （教材P289）

排序方法	最好情况	平均时间	最坏情况	辅助存储	稳定性
简单排序	O(n)	$O(n^2)$	$O(n^2)$	O(1)	稳定
快速排序	O(nlgn)	O(nlgn)	$O(n^2)$	O(lgn)	不稳定
堆排序	O(nlgn)	O(nlgn)	O(nlgn)	O(1)	不稳定
归并排序	O(nlgn)	O(nlgn)	O(nlgn)	O(n)	稳定
基数排序	O(d(n+rd))	O(d(n+rd))	O(d(n+rd))	O(rd)	稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O（1）	不稳定
直接插入	O(n)	$O(n^2)$	$O(n^2)$	O（1）	稳定
折半插入	O(nlgn)	O(nlgn)	O(nlgn)	O（1）	稳定
冒泡	O(n)	$O(n^2)$	$O(n^2)$	O（1）	稳定

1. 简单排序：

   简单选择排序：每次从未排序部分选择最小元素，放到已排序部分的末尾。

   直接插入排序：将每个元素插入到已排序部分的正确位置。

   冒泡排序：通过重复遍历要排序的数列，比较每对相邻元素，如果顺序错误就交换它们。

2. 快速排序：

   选择一个“基准”元素，将数组分为两部分，比基准小往前放，比基准大往后放，然后递归排序这两部分。

3. 堆排序：

   将数组构建成一个堆（通常是大顶堆），堆顶是最大元素，然后将其与数组末尾交换，缩小堆的范围，重新调整堆。

4. 归并排序：

   将数组分成两半，递归地对每一半进行排序，然后将两个已排序的半数组合并成一个完整的有序数组。

5. 基数排序：

   按照位数从低到高依次排序，先按最低位进行桶排序，然后按次低位，依次类推，直到最高位。

这些算法的思想各有特点，适用于不同的场景和数据类型。